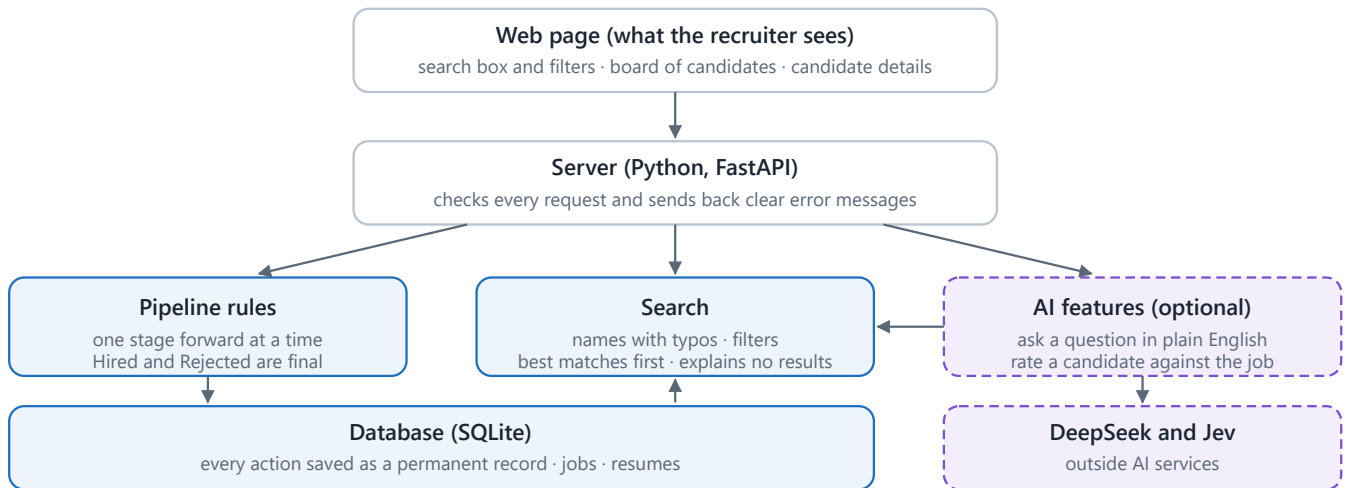


Recruitly · Mini Hiring Pipeline

A simple hiring portal. A recruiter adds candidates, moves them from Applied to Screening, Interview, Offer and Hired, and searches for the people she needs. Every move is saved permanently and can't be changed later.

GitHub repo <https://github.com/Shailly0502/Recruitly>
README [How to run it, the decisions I made and why, what I'd do with more time](#)
Full design [docs/ARCHITECTURE.md](#)

How it's built



The dashed boxes are optional. Without API keys, the AI features are hidden and everything else works normally.

A history that can't be changed

- Every action, like adding, moving or rejecting a candidate, is saved as a **new record**. Old records are never edited.
- The database itself **refuses to edit or delete** records, even if someone skips the app and goes to the database directly.
- Each record is **linked to the one before it** with a fingerprint (a hash). If anyone edits the database file by hand, the app notices.
- If two people act on the same candidate at once, for example a double click, the second action is refused instead of skipping a stage.

Search

- A **search box for names** that allows typos, so "sharam" still finds "Sharma".
- **Filters** for everything else: current stage, stages to leave out, how long someone has been in their stage, when they reached a stage, stages they never reached, and job.
- Filters can be combined. The **best matches come first**.
- Above the results, one line says **exactly what was searched**, for example "Showing candidates who are currently in Screening".
- If nothing matches, it **says why**, for example "Nobody has applied for JOB-002".

The questions from the brief

THE RECRUITER WANTS	HOW SHE GETS IT	THE RECRUITER WANTS	HOW SHE GETS IT
Find Priya Sharma, typed "sharam"	Type it in the search box	Reached Offer but wasn't hired	Reached: Offer, and leave out Hired
Who's in Interview right now?	Stage: Interview	Everyone except rejected	Leave out Rejected
Stuck in Screening for over a week	Stage: Screening, and more than a week	Combine these	Use several filters together
Moved to Interview since Monday	Reached: Interview, since Monday	Something that makes no sense	She's told why there are no results

AI features optional

The app uses two AI services. They do different kinds of work:

- **DeepSeek** is an **LLM**, a model that writes text. I use it when something needs to be written or read: turning a question into a search, and pulling facts out of a resume.
- **Jev** (from TypeSafe) doesn't write text. It **answers a fixed question with a fixed set of answers**, and says how sure it is. For example: "Is this a name or a question?", "Yes or no, and how likely?", or "A score from 1 to 5". I use it to make decisions and give scores, because its answers are easy for the code to check.
- Everything else is **normal code**. The code checks the AI's work before anything is shown or saved.

The AI is only used when the recruiter asks for it: when she types a question and presses Enter, or clicks **Create candidate rating**.

STEP	DEEPSEEK (LLM)	JEV	NORMAL CODE
Search: what did she type?	<i>Not used</i>	Decides if it's a name , a question , off-topic , or unclear	First tries it as a name. The AI is used only if no name matches.
Search: turn the question into a search	Writes the search from her question	<i>Not used</i>	Checks the search is valid (asks DeepSeek to fix it once if not), then fills in the filters
Search: did it understand her?	<i>Not used</i>	Reads the search written back in plain English and says how likely it matches her question	If Jev is at least 70% sure, the search runs. If not, it's shown as a suggestion she can accept or change.
Rating: read the resume	Pulls out skills, jobs and years of experience , each with the exact sentence from the resume that proves it	<i>Not used</i>	Throws away any fact whose sentence isn't really in the resume . Works out years of experience from job dates.
Rating: how good a fit?	<i>Not used</i>	Scores skills, experience and role relevance from 1 to 5 , and how sure it is. It never sees the candidate's name or personal details.	Scores the salary fit , then combines everything into one score out of 10 . Unsure scores are marked "needs review".

Privacy: for search, only the recruiter's question is sent. For ratings, the resume text goes to DeepSeek, but Jev only sees the job requirements and the checked facts. A rating is only advice: it never moves, rejects or hires anyone.

Testing the AI

- **Search:** 30 example questions. An answer counts as correct only if it finds the same candidates as the right answer.

- **Rating:** scores should be close to the expected ones, the same resume should get the same score twice, and **the same resume under a different name should get the same score**, which checks the rating isn't biased by name.

Where I disagreed with the AI

How the AI judges a candidate. The AI's plan was to rate every candidate automatically by comparing their skills and years of experience with the job description. I don't think that's a good way to spot a strong candidate. It only checks whether the right words appear. It can't see how fast someone has grown, how hard or important their work was, skills that carry over from other areas, or whether they'd fit the team. So a resume full of the right keywords could score higher than a better person. Doing this well needs more time, and training on real hiring results. If the recruiter has to double-check every score anyway, it adds work instead of saving it.

What I changed: ratings only run when the recruiter clicks a button. They are advice only and never move anyone. Every fact shows the sentence from the resume it came from. A properly trained scoring model is first on my list below.

What I'd do with more time

- **A proper ATS scoring model**, trained on real hiring results and on recruiters' feedback, and regularly checked for bias.
- **A better database**, with separate tables for candidates, jobs, applications and history, properly linked to each other. It would move to PostgreSQL so it can handle many more candidates.
- **More features in the interface:** drag and drop on the board, acting on several candidates at once, saved searches, notes and tags, interview scheduling and email templates.
- **A practice area for recruiters**, with made-up candidates, where they can try searches and rating settings without touching real data.
- **Reports and alerts**, such as time to hire, how many people pass each stage, and warnings like "5 candidates have been stuck in Screening for over a week".
- Searching inside resumes by meaning, AI-suggested interview questions, spotting duplicate candidates, and user accounts that record who did what.

Tech stack and how to run it

Built with Python, FastAPI and SQLite, and a plain HTML, CSS and JavaScript front end. DeepSeek and Jev are optional.

To run it: install the requirements with `pip install -r requirements.txt`, start it with `uvicorn app.main:app`, open `localhost:8000`, and click **Load demo candidates**.